Zurich University of Applied Sciences

School of Management and Law

Prototyping – Vorlesung 4
SvelteKit Teil 2: Objekte, Databinding, Logik



Building Competence. Crossing Borders.

Max Meisterhans
max.meisterhans@zhaw.ch

Inhalt

- JavaScript-Objekte
- Darstellung von Listen
- Conditional Rendering
- Pfeilfunktionen



Objekte

Objects – Containers for properties

Each property has a name ("key") and a value:

```
let person = {
    firstName: "Luke",
    lastName: "Skywalker"
};
```

Objects can be nested:

```
let flight = {
    airline: "Oceanic",
    number: 815,
    departure: {
        IATA: "SYD",
        time: "2004-09-22 14:55",
    },
    arrival: {
        IATA: "LAX",
        time: "2004-09-22 10:42",
    }
}
```

Accessing properties

Two ways of accessing values of objects:

- By using the bracket notation "[ ]"

```
person["lastName"] // "Skywalker"
```

- By using the dot notation "."

```
flight.departure.IATA // "SYD"
```

- Or a combination of both ways

```
flight["departure"].IATA // "SYD"
```

Accessing properties

The bracket notation "[ ]" must be used if the key is a variable:

```
let person = {
  firstName: "Luke",
  lastName: "Skywalker",
};

let myKey = "lastName";

console.log(person.myKey); // output: undefined

console.log(person[myKey]); // output: Skywalker
```

Change property values

The value of property can be changed by accessing it by its name:

```
person["firstName"] = "Anakin";
flight.number = 800;
```

If the property does not exist yet, it will be added to the object.

```
person["nickname"] = 'Darth Vader';
flight.equipment = {
    model: 'Boeing 777'
};
flight.status = 'overdue';
```

Kurzübung – Zugriff auf Objekteigenschaften

```
let planet = {
    name: "Mars",
    surfaceTemperatureCelsius: {
        min: -143,
        mean: -63,
        max: 35
    },
    satellites: [
        {
            name: "Phobos",
            discoveryDate: "18 August 1877"
        },
        {
            name: "Deimos",
            discoveryDate: "12 August 1877"
        }
    ]
}

let discoverers = {
    Phobos: "Asaph Hall",
    Deimos: "Asaph Hall",
    Ganymede: "Galileo Galilei"
}
```

Ergänze den folgenden Code, damit die gewünschte Ausgabe auf die Konsole geschrieben wird.

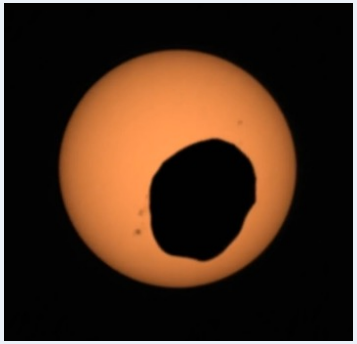
```
// Datum der Entdeckung von Phobos
console.log( planet. ? );

// Temperaturrange auf dem Mars (Maximum – Minimum)
console.log(   planet. ?
             - planet. ? );

// Name des Entdeckers von Phobos
console.log( discoverers[planet ? ] );
```

Erwarteter Output:

```
18 August 1877
178
Asaph Hall
```



Databinding, Lists

Transformation: From JavaScript Objects to HTML

```
[
  { country: "Aruba", capital: "Oranjestad"},
  { country: "Afghanistan", capital: "Kabul"},
  { country: "Angola", capital: "Luanda"},
  { country: "Anguilla", capital: "The Valley"},
  { country: "Albania", capital: "Tirana"},
  { country: "Andorra", capital: "Andorra la Vella"}
]
```

Data (JS Objects or JSON)

Synchronization?

HTML

Country	Capital
Aruba	Oranjestad
Afghanistan	Kabul
Angola	Luanda
Anguilla	The Valley
Albania	Tirana
Andorra	Andorra la Vella

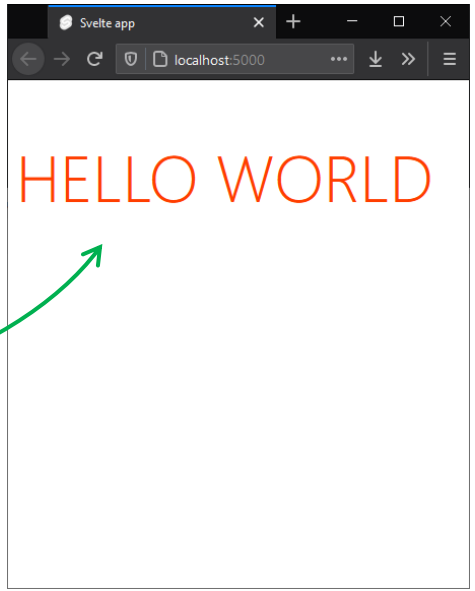
Recap: Databinding

Svelte uses curly brackets { .. } to insert JavaScript code, in particular *variables*, into HTML.

```
<script>
    let message =  $state("Hello World");
</script>

<h1>{message}</h1>

<style>
    h1 {
        color: #ff3e00;
        text-transform: uppercase;
        font-size: 4em;
        font-weight: 100;
    }
</style>
```



Lists with “Each blocks”

“Each blocks” can be used to render lists of items. This is especially useful for generating lists and tables in HTML. *(See also https://svelte.dev/tutorial/svelte/each-blocks)*

```
<script>
    let todos = $state([
        { text: "Get a lightsaber" },
        { text: "Use the force" },
        { text: "Destroy the Death Star" }
    ]);
</script>

<ul>
    {#each todos as todo}
        <li>{todo.text}</li>
    {/each}
</ul>
```

Transformation →

- Get a lightsaber
- Use the force
- Destroy the Death Star

```
<ul>
    <li>Get a lightsaber</li>
    <li>Use the force</li>
    <li>Destroy the Death Star</li>
</ul>
```

Kurzübung – Databinding, Each-Blöcke

a) Öffne folgenden Playground:
https://svelte.dev/playground/860710bef19f46079eb13eefb9f7ade8?version=5.1.16

b) Füge der App einen Titel hinzu:
   a) Erstelle eine Variable `title` in `<script>` und weise einen Wert zu, z.B. “My first Svelte App”.
   b) Verwende die Variable `title` im HTML-Teil der App (mit geschweiften Klammern)

c) Verwende einen `#each`-Block, um eine Liste von Filmen anzuzeigen:
   a) Erstelle ein Array `movies` in `<script>`, z.B. `["Star Wars IV", "Star Wars V", "Star Wars VI"]`
   b) Erstelle eine unordered list `<ul>` und verwende einen #each-Block um die Filme als `<li>`-Elemente darzustellen.



Logic, Functions, Value Bindings

Reactivity

Reactivity means that the page will automatically be kept in sync with the application state. To make variables reactive, we need to wrap it with `$state(…)`:

https://svelte.dev/tutorial/svelte/state https://svelte.dev/docs/svelte/what-are-runes

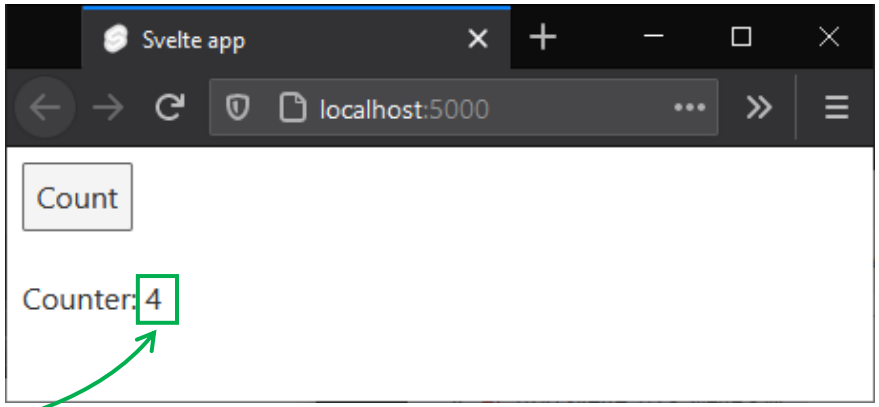
Initial value

```
<script>
    let counter = $state(0);

    function count() {
        counter += 1;
    }
</script>

<button onclick={count}>Count</button>

<p>Counter: {counter}</p>
```



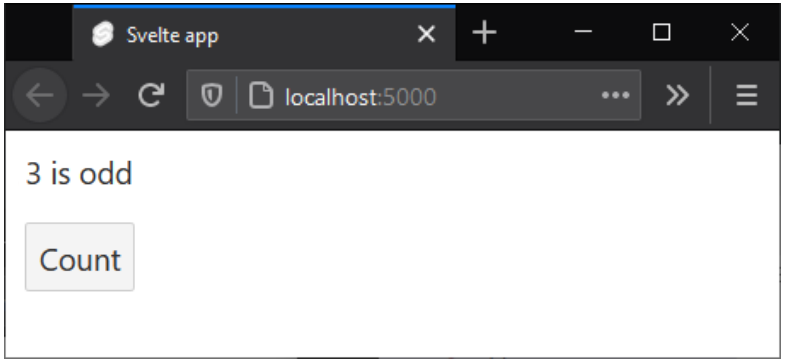
if – else if – else

Svelte offers special blocks for conditionally rendered content, see https://svelte.dev/docs/svelte/if

```
<script>
    let counter = $state(0);

    function count() {
        counter += 1;
    }
</script>

<p>{counter} is
  {#if counter % 2 === 0}
    <span>even</span>
  {:else}
    <span>odd</span>
  {/if}
</p>
<button onclick={count}>Count</button>
```



Reactivity: Working with Arrays and Objects

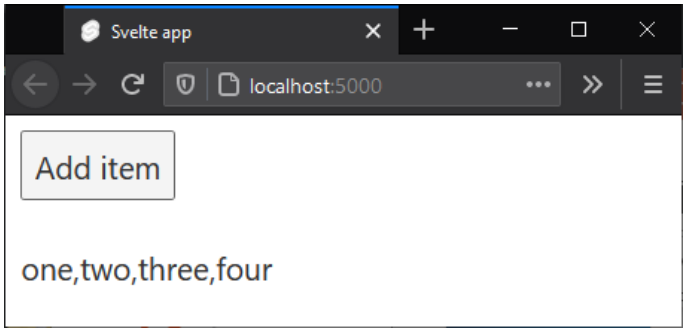
Use **`$state(...)`** if the array should be kept in sync with the HTML.

```
<script>
    let items = $state(["one","two","three"]);

    function addItem() {
        items.push("four"]);
    }
</script>

<button onclick={addItem}>Add item</button>

<p>{items}</p>
```



Value Binding

Use the `bind:value` directive to “connect” the value of an input field to a variable. See also https://svelte.dev/tutorial/svelte/text-inputs

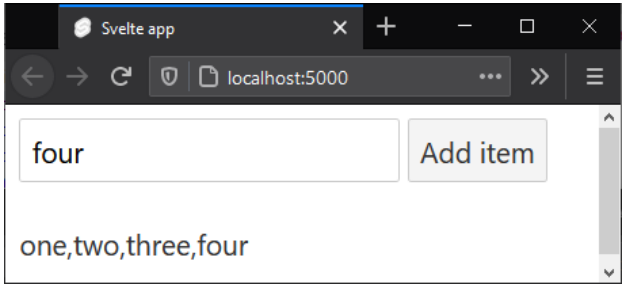
Updating works in both directions:

```
<script>
    let items = $state(["one","two","three"]);
    let input = $state("");
    function addItem() {
        items.push(input);
    }
</script>

<input type="text" bind:value={input}>
<button onclick={addItem}>Add item</button>

<p>{items}</p>
```

If the value of the variable “input” changes, the value of the input field will be updated.



If a user enters data, the variable “input” will be updated.

Kurzübung – Funktionen, Value Binding

a) Öffne das folgende REPL:
https://svelte.dev/playground/a6f605b689ec42159741b816357a0d0c?version=5.1.16

b) Füge ein input-Element und einen Button “Add Movie” zum HTML-Teil der App hinzu. Bei Klick auf den Button soll die Funktion `addMovie()` aufgerufen werden. (Diese Funktion wird erst im letzten Schritt implementiert.)

c) Erstelle eine Variable `movie` und verknüpfe diese mit dem input-Element. Nutze dazu `bind:value={...}`

d) Implementiere die Funktion `addMovie()` im `<script>`-Teil. Diese Funktion soll den im input-Element eingegebenen Film in das Array `movies` hinzufügen.
Nutze `$state(...)` für das Array `movies`, damit das HTML bei Wertänderungen aktualisiert wird.

e) (Optional) Deaktiviere das Inputfeld und den Button «Add Movie», wenn 5 oder mehr Filme in der Liste sind. Dazu brauchst du das Attribut disabled https://www.w3schools.com/tags/att_disabled.asp, das dynamisch sein muss, siehe https://svelte.dev/tutorial/svelte/dynamic-attributes





Pfeilfunktionen (Arrow Functions)

JavaScript has "First-class functions"

Functions in JavaScript can be used like any other variable. A function can be:

- assigned as value to a variable.
- passed as parameter to another function.
- be returned by another function.

```
function add(x, y) {
  return x + y;
}

function multiply(x, y) {
  return x * y;
}

function calculate(f, x, y) {
  let result = f(x, y);
  console.log(result);
}

calculate(add, 2, 3); // 5

let m = multiply; // assign to a variable
calculate(m, 2, 3); // 6
```

See https://developer.mozilla.org/en-US/docs/Glossary/First-class_Function

Arrow functions – A compact alternative to regular functions.

Arrow functions are a compact alternative to regular (anonymous) functions. They are particularly useful for passing functions as parameters.

Basic arrow function:

```
(param1, param2, …, paramN) => { statements }
```

Arrow function with only one expression as return value:

```
(param1, param2, …, paramN) => expression // "Expression body"
// equivalent to:
(param1, param2, …, paramN) => { return expression; } // "Block body"
```

For a function with no parameters, use a pair of parentheses:

```
() => { statements }
```

Arrow function have more advantages. See https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions for more information.

Arrow functions – Examples

```
function add(x, y) {
  return x + y;
}

function multiply(x, y) {
  return x * y;
}

function calculate(f, x, y) {
  let result = f(x, y);
  console.log(result);
}

calculate(add, 2, 3);      // 5
calculate(multiply, 2, 3); // 6
```



```
function calculate(f, x, y) {
  let result = f(x, y);
  console.log(result);
}

// With "block body"
calculate((x, y) => { return x + y; }, 2, 3); // 5

// With "expression body"
calculate((x, y) => x * y , 2, 3); // 6
```

Block body
Explicit return statement.

Expression body
Implicit return statement.

Why do we need all this?

In some cases, functions are only be executed in case of an event firing, e.g., when a button is clicked or when we receive data from a server.

Example – `setTimeout(…)`:

```
function hello() {
  alert("Hello World!");
}

setTimeout(hello, 5000);
```

`setTimeout()` has 2 parameters:

- A function, a so-called "callback function"
- Time in milliseconds

→ The function `hello()` will be executed after 5 seconds.

Anonymous functions as callback functions

In the previous example, we used the function `hello()` only once. Therefore, it is not necessary to give this function a name and we can use an anonymous function instead:

With a normal anonymous function:

```
setTimeout( function () {
    alert('Hello World!');
}, 5000);
```

With an arrow function:

```
setTimeout( () => {
    alert('Hello World');
}, 5000)
```

Summary

```
// normal function
function add(x, y) {
  return x + y;
}
```

```
// anonymous function as value of a variable
var add = function (x, y) {
  return x + y;
};
```

```
// arrow function as value of a variable
var add = (x, y) => { return x + y; }
var add = (x, y) => x + y; // equivalent
```

```
// functions as parameters
function calculate(f, x, y) {
  console.log(f(x, y));
}
calculate(add, 1, 2); // output: 3
```

Kurzübung – Pfeilfunktionen als Parameter

Welchen Output erzeugt der folgende Code?

```
function calculate(f1, f2, x, y) {
  let a = f1(x, y);
  if (a > 2) {
    return f2(y);
  }
  return f2(x - 5);
}

let result = calculate(
  (x, y) => x / y,
  (x) => {
    x++;
    return x * x;
  },
  10,
  5
);

console.log(result);
```